

# CS 546

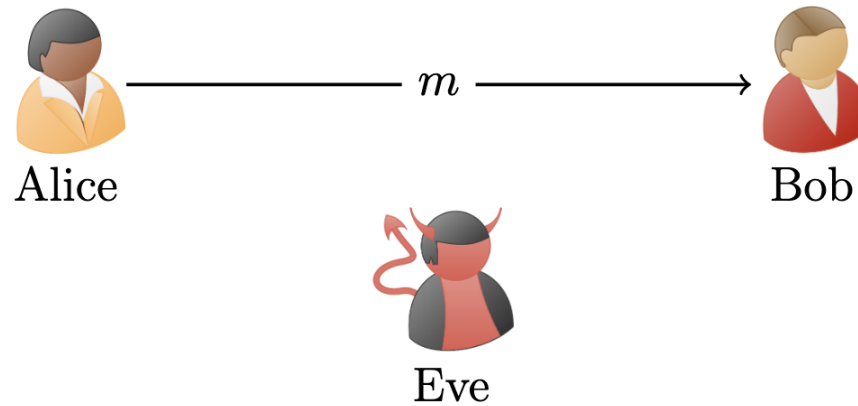
## Computer System Security

### Lecture 3.2: Secure Messaging

Instructor: Aarushi Goel

Fall 2026

# Secure Messaging Goals



- **Confidentiality:** Only Alice and Bob can read the message
- **Integrity:** Bob knows Eve has not tampered with the message (and that it has not been corrupted)
- **Authentication:** Bob knows Alice wrote the message

# Solution to Secure Messaging

## End-to-End Encryption



The server is still necessary for routing and temporary storage. End-to-end encryption means the server transports ciphertext without possessing the conversation keys.

## Other Requirements

### Confidentiality

The server and network should not learn the plaintext message.

### Asynchrony

Alice must be able to start a conversation while Bob is offline.

### Long-term security

Compromise today should not automatically reveal all past or all future messages.

# How Can Alice and Bob Share a Common Key?

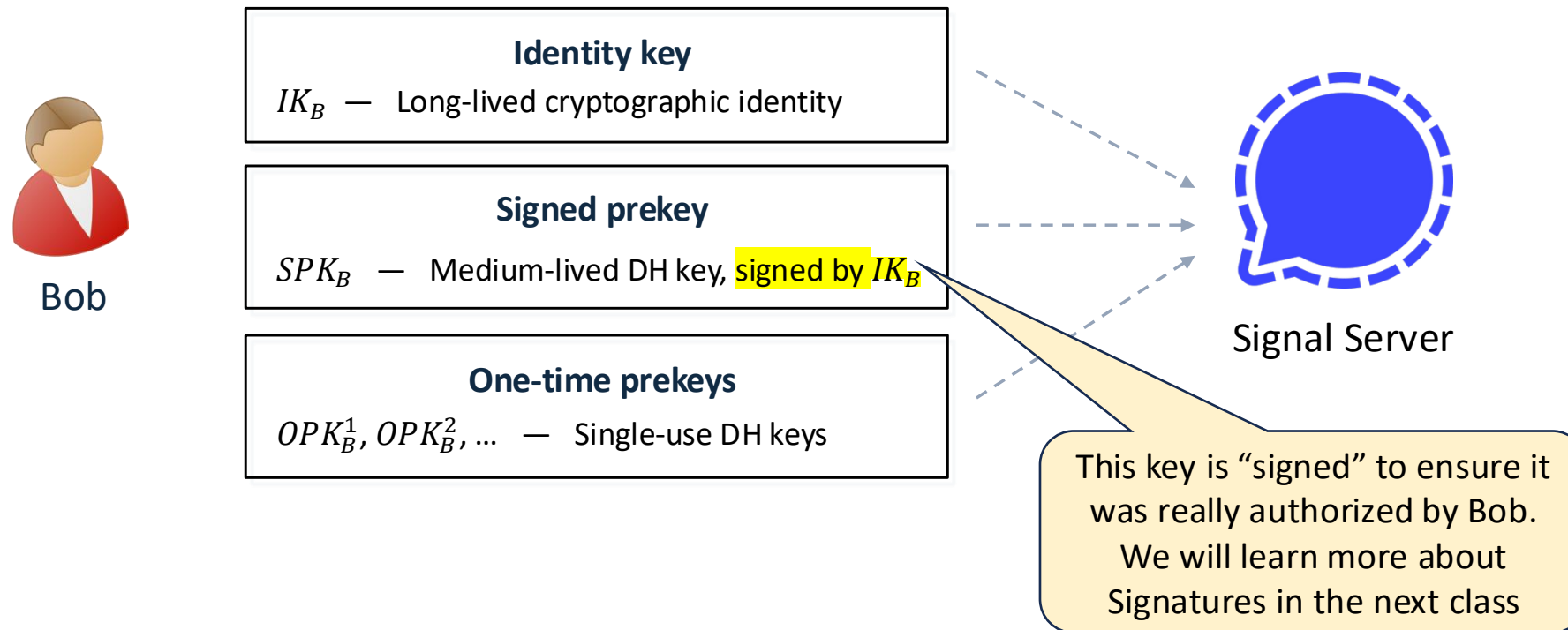
## Why does regular key-exchange not suffice?

- Regular key-exchange, e.g., the Diffie-Hellman protocol requires both parties to participate at the same time.
- But what if Bob's phone is offline when Alice wants to send him a message.
- **Signal's Idea:** Bob publishes public key material in advance. Alice can use that material to establish a session now; Bob reconstructs the same secret when he comes back online.

# Signal's Key-Exchange Idea

## Bob publishes prekeys

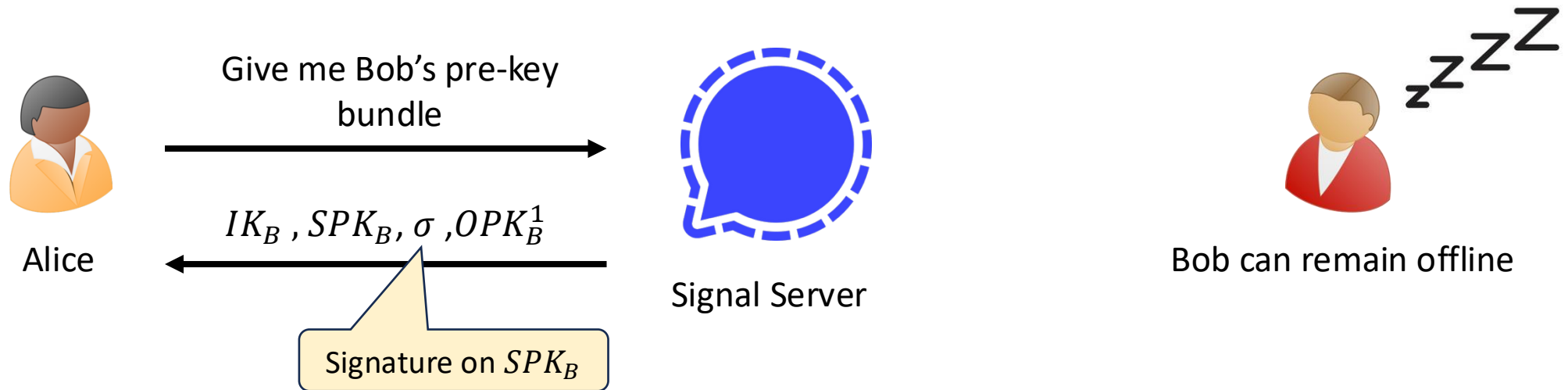
Bob prepares public key material before anyone contacts him.



Only the public key parts are sent. Bob keeps the corresponding private keys on his device.

# Signal's Key-Exchange Idea

Alice starts a session



# Signal's Key-Exchange Idea

## Alice starts a session



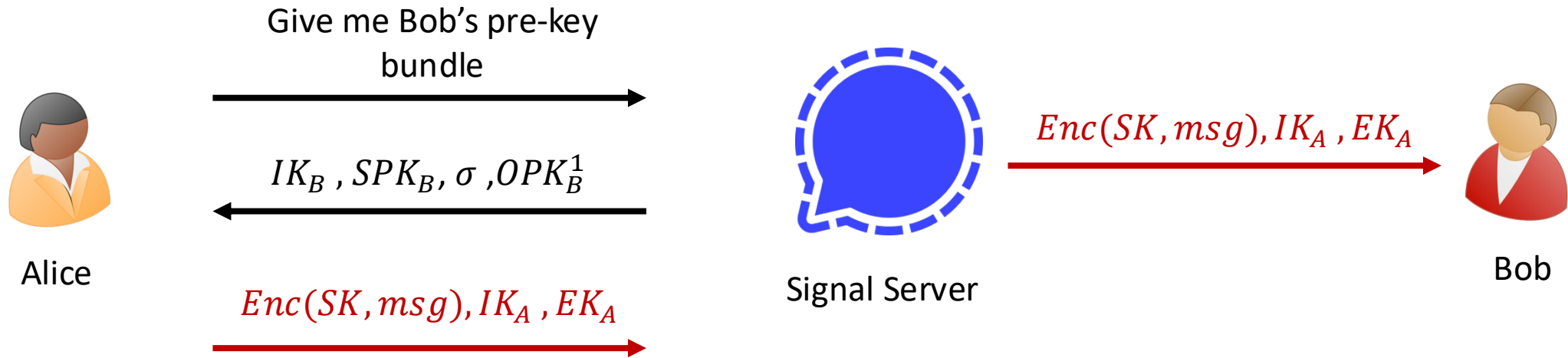
Alice then does the following:

- Verifies if  $\sigma$  is a valid signature on  $SPK_B$  using Bob's identity key  $IK_B$ .
- Samples a random ephemeral Diffie-Hellman key  $EK_A = g^a$ .
- Uses the Diffie-Hellman key-exchange to obtain the following keys:
  - $DH_1 = DH(IK_A, SPK_B)$ : bind Alice's identity to Bob's signed prekey
  - $DH_2 = DH(EK_A, IK_B)$ : bind Alice's fresh ephemeral key to Bob's identity
  - $DH_3 = DH(EK_A, SPK_B)$ : fresh session contribution
  - $DH_4 = DH(EK_A, OPK_B^1)$ : one-time contribution
- Finally, it runs a key-derivation function (KDF) to combine these keys and derive a secret encryption key

$$SK = KDF(DH_1, DH_2, DH_3, DH_4)$$

# Signal's Key-Exchange Idea

Alice starts a session



# Signal's Key-Exchange Idea

## When Bob comes online



When Bob comes online and receives  $Enc(SK, msg), IK_A, EK_A$  from Signal:

- Bob reconstructs the same  $SK$  by first running the Diffie-Hellman key-exchange on his private pre-keys and Alice's public-key information and then running the key-derivation function.
- After use, the one-time key  $OPK_B^1$  can be deleted.

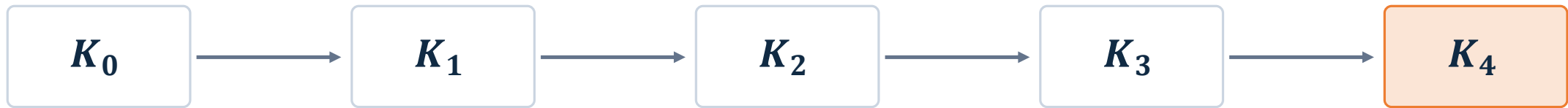
Can Alice and Bob now continue using the same  $SK$  to exchange encrypted messages with each other?

**No!!**

If an attacker later steals  $SK$ , all recorded ciphertexts could potentially be decrypted. Signal therefore doesn't treat  $SK$  as the permanent encryption key. Instead  $SK$  becomes the starting point for the **Double Ratchet**.

# Signal's Double Ratchet Idea

## What is a Ratchet?



$$K_{i+1} = \text{Hash}(K_i)$$

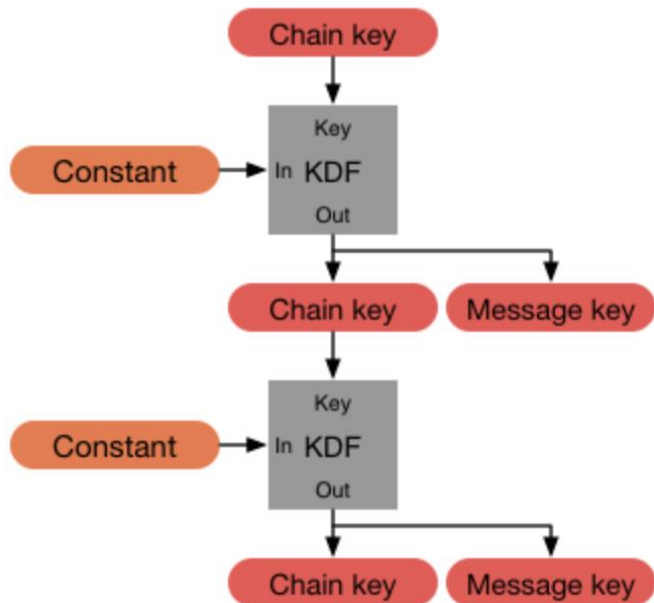
We will learn more about hash functions in the next lecture.

- **Forward direction:** Knowing  $K_3$  lets you compute  $K_4$ .
- **Backward direction:** Knowing  $K_4$  should not let you recover  $K_3$ ,  $K_2$ , ...

Delete old states after advancing. This deletion is important for security.

# Signal's Double Ratchet Idea

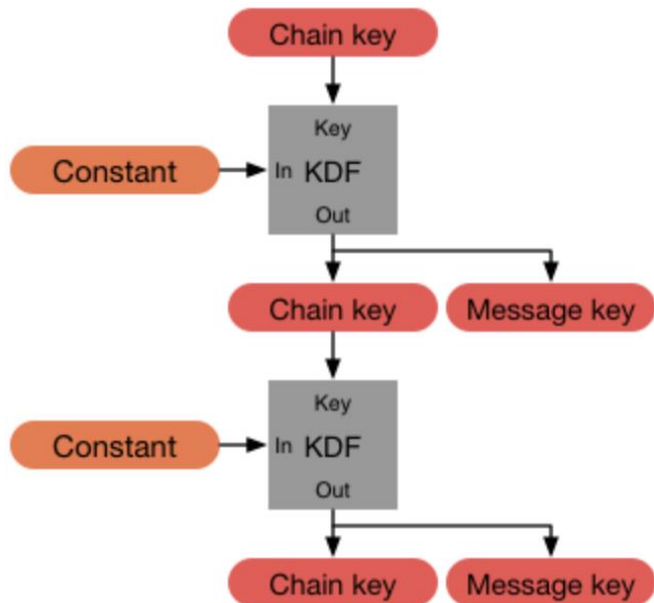
## Ratchet #1: Symmetric-Key Ratchet



- Suppose Alice has a sending chain key:  $CK_0$ .
- When she wants to send a message, she computes  $KDF(CK_0) = (CK_1 MK_0)$ . The KDF gives two outputs: a **new chain key** and a **message key**.
- Alice encrypts using the derived message key:  $Enc_{MK_0}(M_0)$ .
- Then she deletes  $CK_0$ .
- Whenever she wants to send the next message she computes  $KDF(CK_1) = (CK_2 MK_1)$  and uses  $MK_1$  to encrypt the next message.
- This way, every message is encrypted under a new key.

# Signal's Double Ratchet Idea

## Ratchet #1: Symmetric-Key Ratchet



### Good:

- Suppose an attacker compromises Alice when her chain is at  $CK_{100}$ .
- The attacker can compute  $CK_{101}, CK_{102}, \dots$ , but should not be able to go backward and find  $CK_{99}$ .
- If old message keys were erased, the attacker cannot recover  $MK_{99}, MK_{98}, \dots$
- This gives forward secrecy for old messages.

### Bad:

- Suppose the attacker steals  $CK_{100}$ .
- Then they can compute  $CK_{101}, CK_{102}, CK_{103}, \dots$ , because the chain moves deterministically forward.
- This could expose future messages too.

# Signal's Double Ratchet Idea

## Ratchet #2: Diffie-Hellman Ratchet

- We need some way to inject **new secret randomness** that the attacker doesn't know.
- That is injected via a **second ratchet**.
- Alice and Bob periodically generate fresh DH key pairs.
- Suppose Alice currently has  $(a, A)$  and Bob has  $(b, B)$ .
- Alice and Bob can derive  $DH(a, B) = DH(b, A)$ .
- This fresh DH secret gets mixed into the protocol.

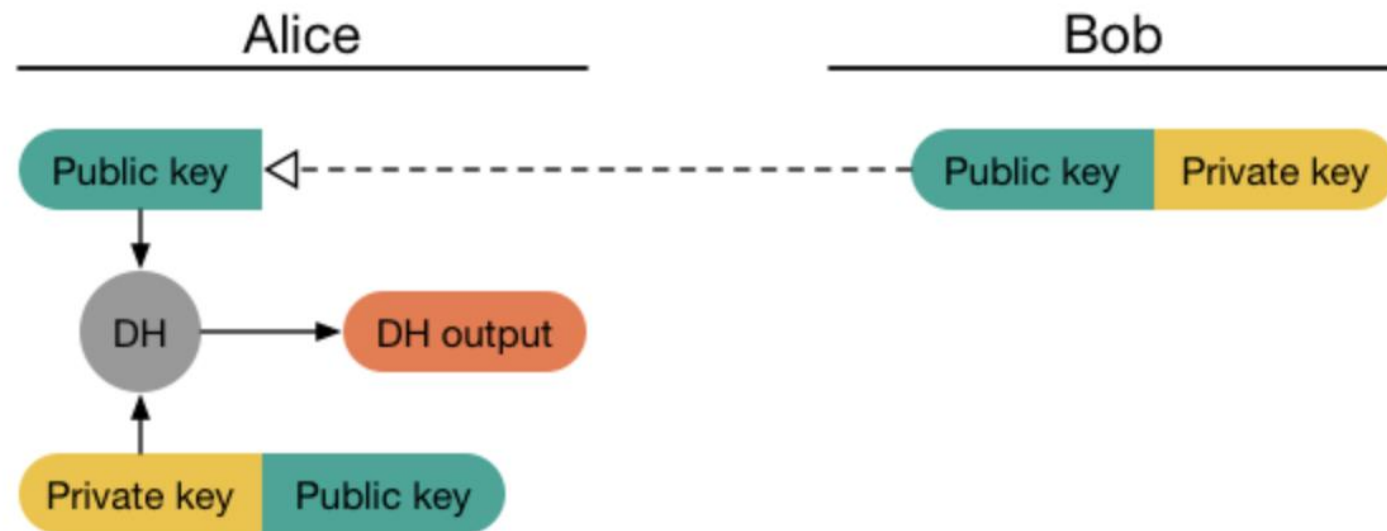
If an attacker knew the old state but later loses access to Alice's and Bob's devices, a fresh DH exchange can introduce a secret the attacker does not know.

# Signal's Double Ratchet Idea

## Ratchet #2: Diffie-Hellman Ratchet

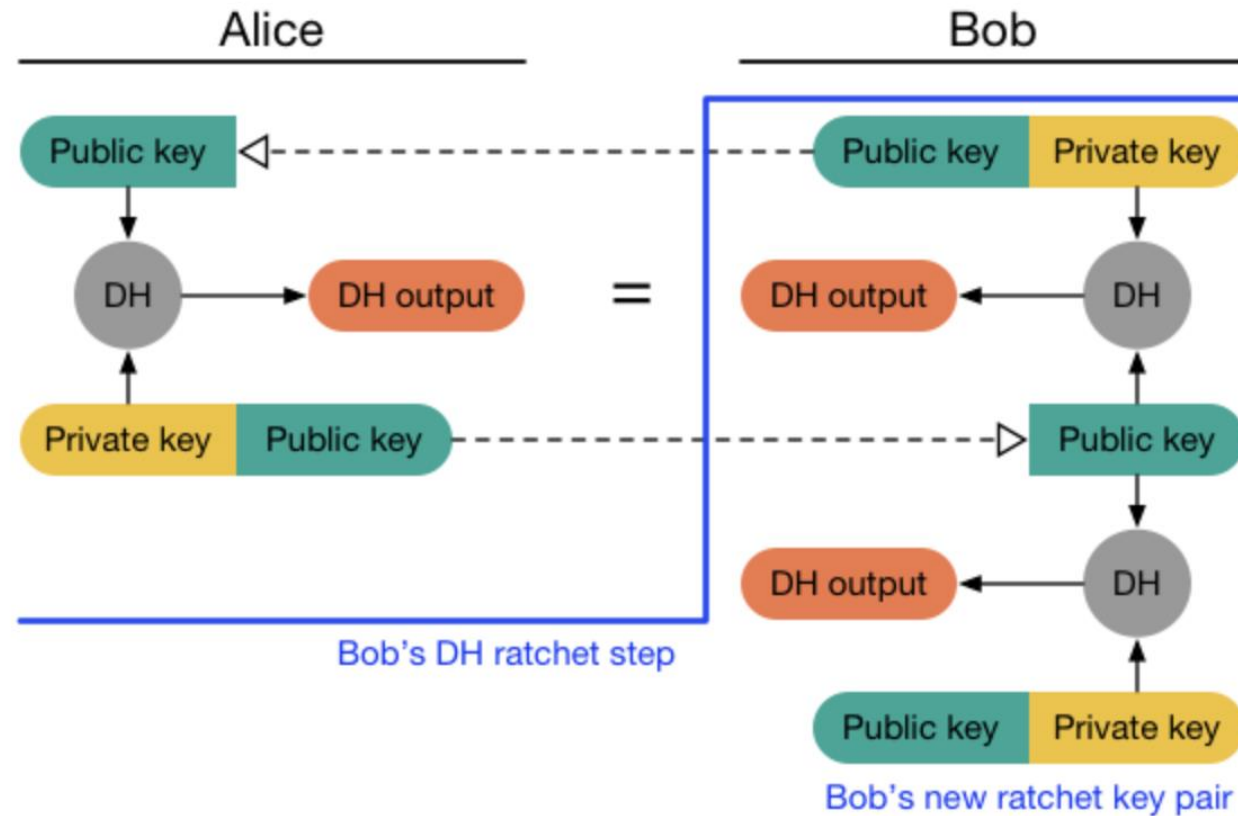
- The original  $SK$  that was generated at the start of the session becomes a root key  $RK_0$  for Alice to send messages.
- She samples a new DH key  $g^{a_1}$  and computes  $KDF(RK_0, g^{a_1}, a_1) = (RK_1, CK_0)$ .
- She deletes  $RK_0$  and uses  $CK_0$  to initialize a sending chain. She continues to use Ratchet #1 (symmetric-key ratchet) to send messages to Bob, **until he sends a message**.
- Bob uses his private key material and Alice's public key material to derive the same  $(RK_1, CK_0)$  and decrypt messages sent by Alice.
- If Bob replies, he will sample and send a new DH key  $g^{b_1}$ . It uses this to derive  $KDF(RK_1, g^{b_1}, b_1) = (RK_2, CK'_0)$ . He deletes all previous keys.
- He now derives message keys using Ratchet #1 on  $CK'_0$  to send messages to Alice.
- The next Diffie-Hellman ratchet takes place when the conversation switches directions and a new DH public key arrives

# Signal's Double Ratchet Idea



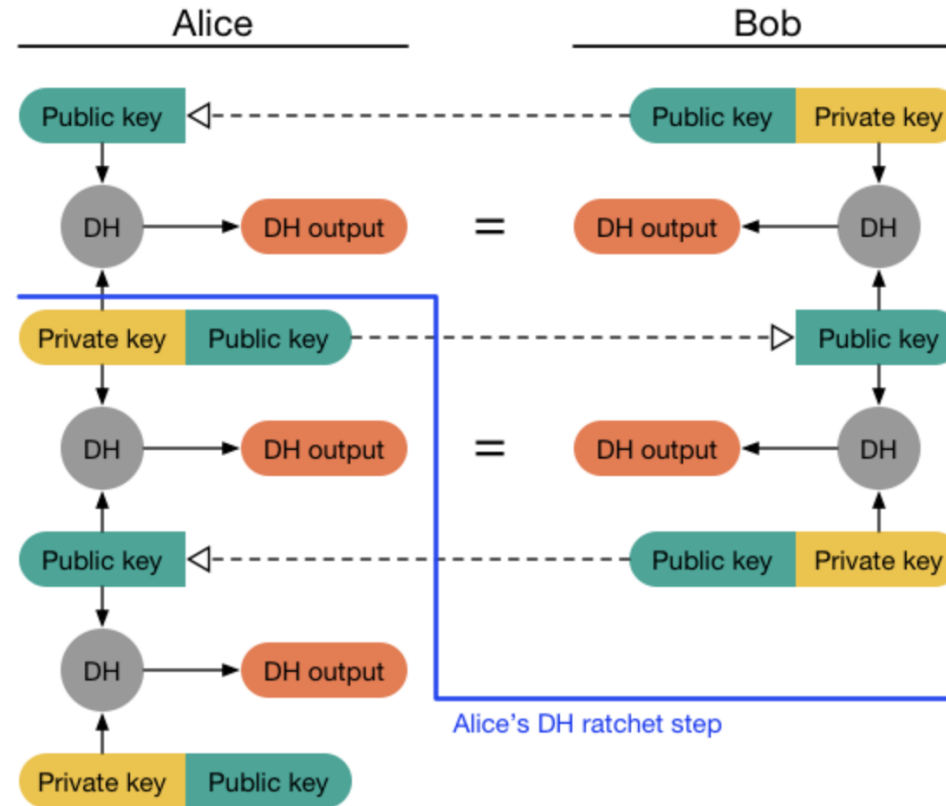
Alice is initialized with Bob's ratchet public key. Alice's ratchet public key isn't yet known to Bob. As part of initialization Alice performs a DH calculation between her ratchet private key and Bob's ratchet public key.

# Signal's Double Ratchet Idea



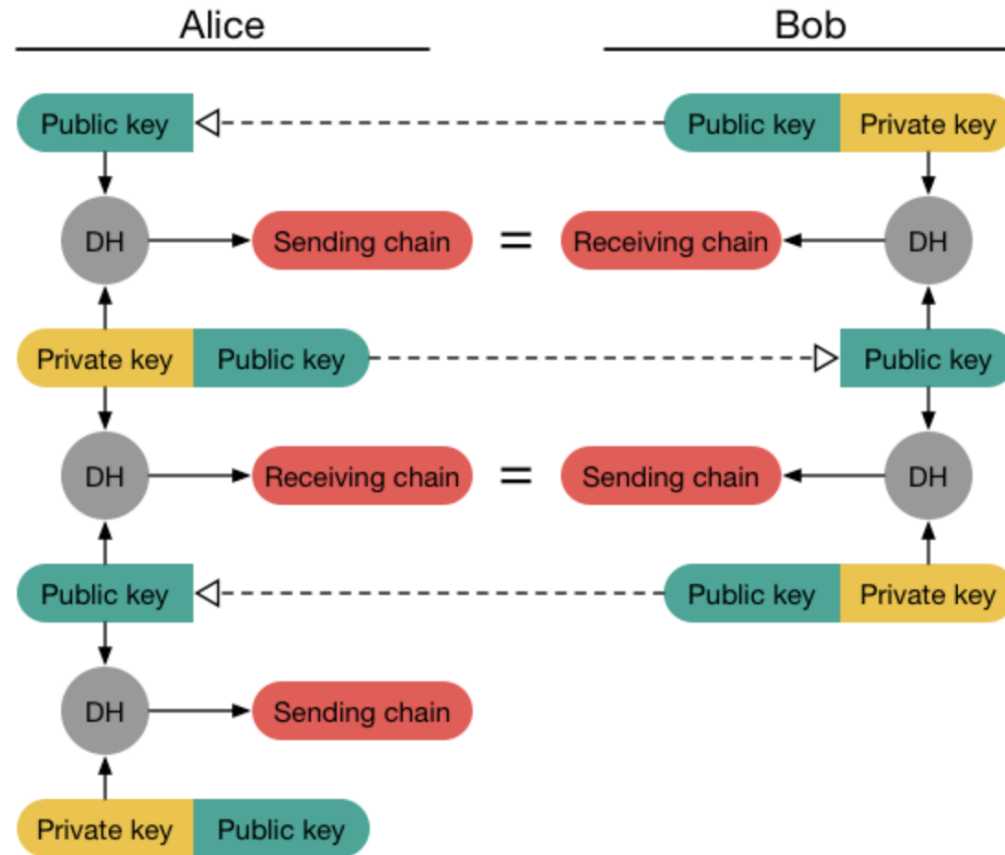
Alice's initial messages advertise her ratchet public key. Once Bob receives one of these messages, Bob performs a DH ratchet step: He calculates the DH output between Alice's ratchet public key and his ratchet private key, which equals Alice's initial DH output. Bob then replaces his ratchet key pair and calculates a new DH output.

# Signal's Double Ratchet Idea



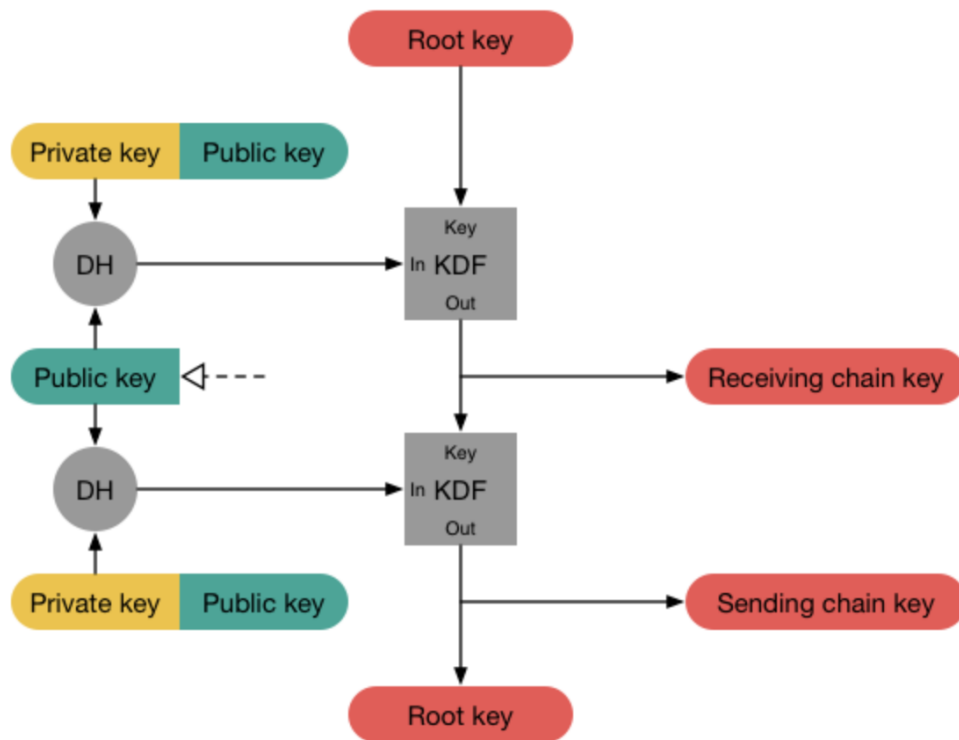
Messages sent by Bob advertise his new public key. Eventually, Alice will receive one of Bob's messages and perform a DH ratchet step, replacing her ratchet key pair and deriving two DH outputs, one that matches Bob's latest and a new one.

# Signal's Double Ratchet Idea



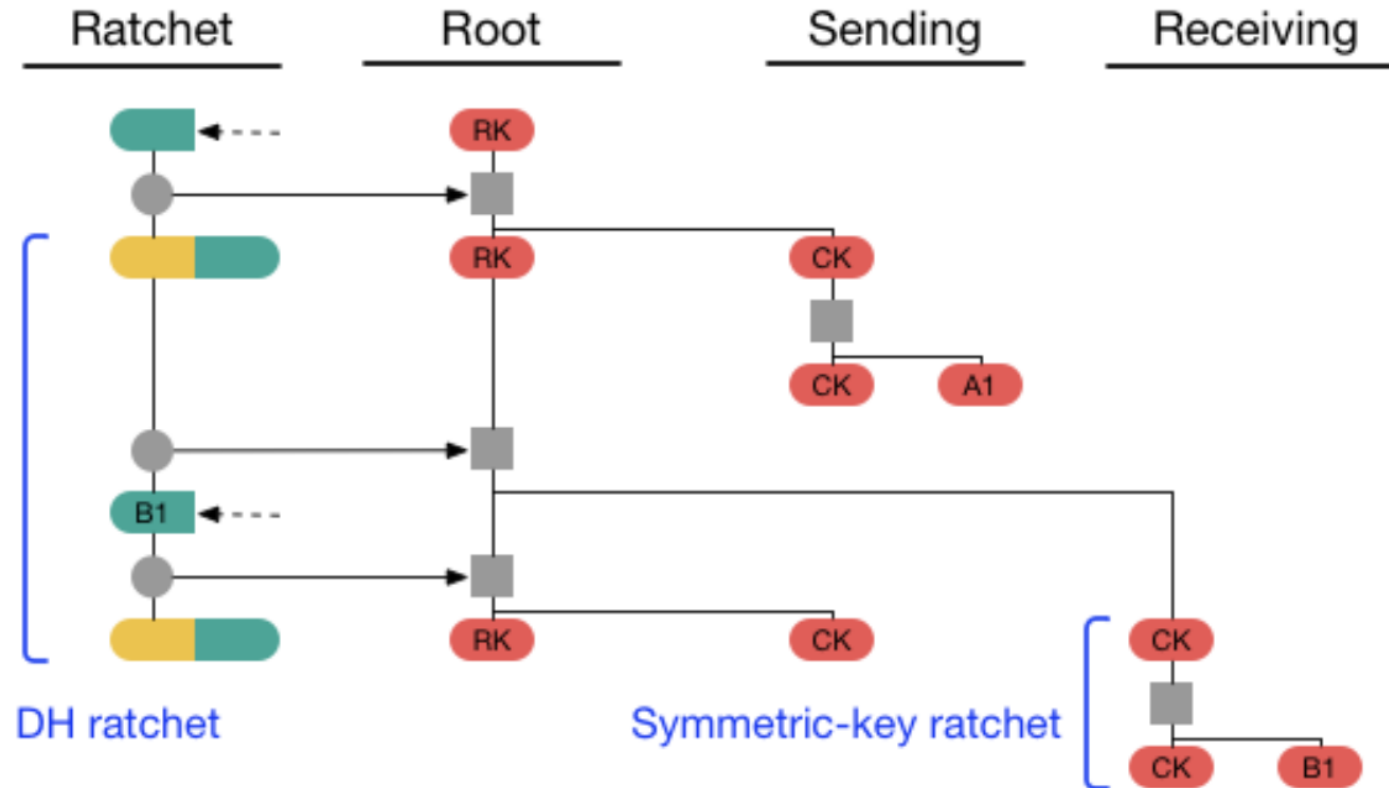
The DH outputs generated during each DH ratchet step are used to derive new sending and receiving chain keys. The below diagram revisits Bob's first ratchet step. Bob uses his first DH output to derive a receiving chain that matches Alice's sending chain. Bob uses the second DH output to derive a new sending chain.

# Signal's Double Ratchet Idea



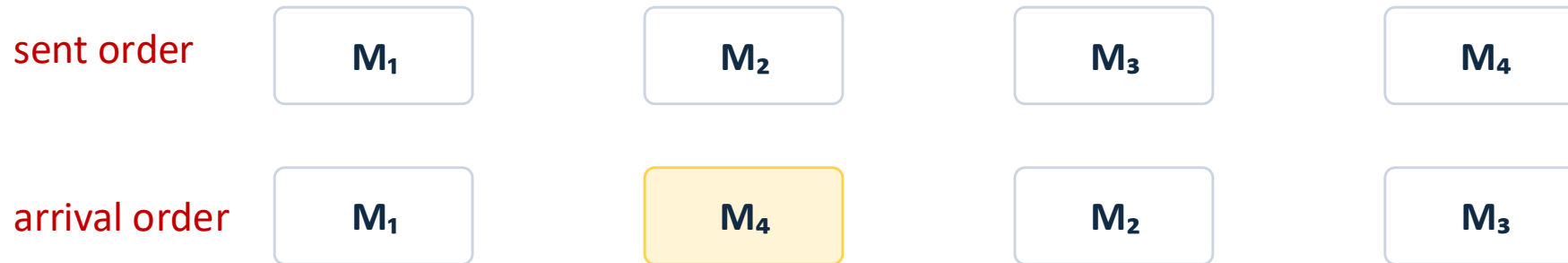
- Whatever we have discussed so far, is a simplification. Instead of taking the chain keys directly from DH outputs, the DH outputs are used as KDF inputs to a root chain, and the KDF outputs from the root chain are used as sending and receiving chain keys. Using a KDF chain here improves resilience and break-in recovery.
- A full DH ratchet step consists of updating the root KDF chain twice, and using the KDF output keys as new receiving and sending chain keys:

# Signal's Double Ratchet Idea

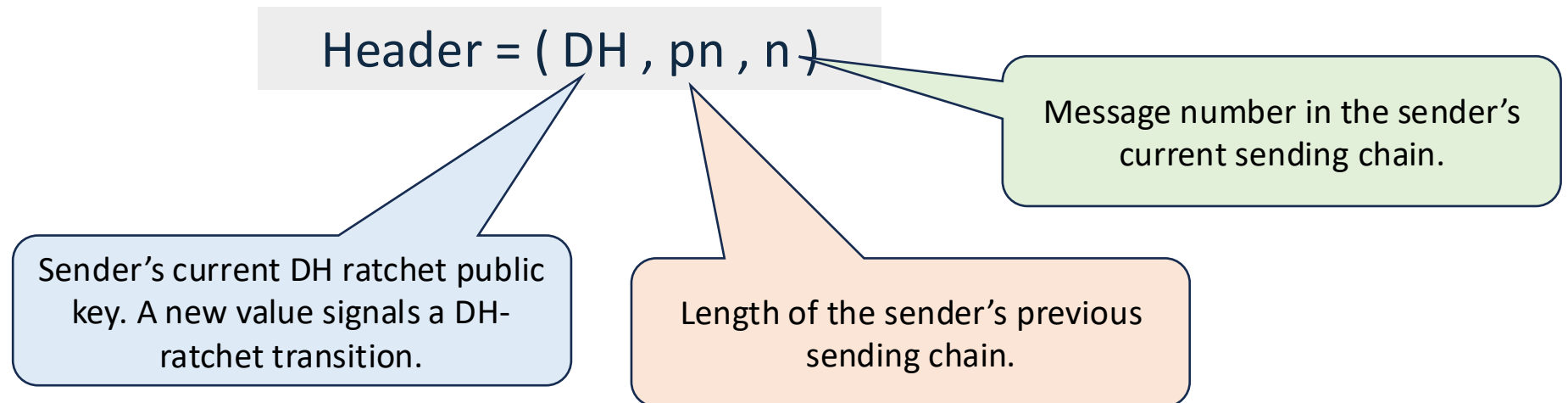


# Out-of-order delivery

Signal must work even when messages arrive as  $M_1, M_4, M_2, M_3$ .

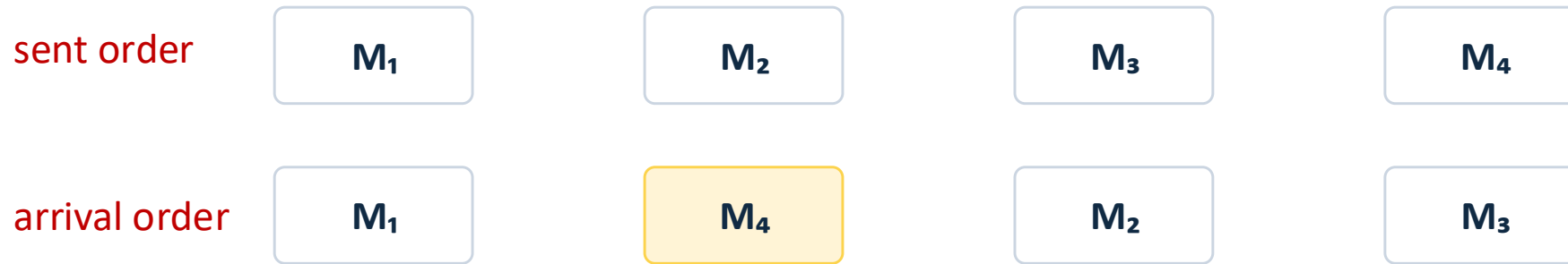


Each message must carry enough information to synchronize the receiver.



# Out-of-order delivery

Signal must work even when messages arrive as  $M_1, M_4, M_2, M_3$ .



## When $M_4$ arrives early

The header says  $n = 4$ . Alice advances her receive chain enough times to derive  $MK_2$ ,  $MK_3$ , and  $MK_4$ .

## What Alice stores

Use  $MK_4$  now; temporarily cache the skipped keys  $MK_2$  and  $MK_3$  for the delayed messages. Importantly, she stores specific skipped message keys—not the old chain key.

# Forward-Secrecy vs Post-Compromise Security

## Forward Secrecy:

- Compromise of current state should not reveal old messages whose keys were already erased.
- Main mechanism: one-way symmetric-key evolution + deletion.

## Post-Compromise Security:

- If an attacker compromises a device but later loses access, fresh DH randomness can eventually make future messages secret again.
- Main mechanism: Diffie-Hellman ratchet.

# Other Features in Signal

- Post-quantum security.
- Group messaging.
- Sending encrypted attached files.
- Multi-device support.
- Authenticating the identity key: the protocol that we have discussed let's Alice check authenticity w.r.t. an identity key, but how does Alice know that key really belongs to Bob?